Machine Learning & Deep Learning

Interview Questions and Answers

Part 3 — MLOps, Scenarios & Coding

Fundamentals • Statistics • Classical ML • Evaluation • Deep Learning • CNNs • RNNs • Transformers • NLP • MLOps • Scenarios • Coding

Table of Contents

19. MLOps, Deployment & Efficiency
20. Scenario-Based & System Design Questions
21. Coding Questions (Implement from Scratch)

Study Roadmap: Beginner to Advanced

Within every section, questions go roughly from basic to advanced. Suggested order:

Level	Focus	Sections
1. Beginner	Core vocabulary, types of ML, basic statistics, data cleaning, linear/logistic regression, decision trees, basic metrics, neural network basics, Python libraries	1, 2 (first half), 3, 4, 6 (first half), 9 (first half), 13 (first half), 21 (metrics, pandas, linear regression)
2. Intermediate	Optimization, ensembles and boosting, SVM/KNN/Naive Bayes, clustering and PCA, tuning, time series, training deep nets, CNNs, RNNs, NLP	5, 6, 7, 8, 10, 11, 13–16, 18, 21 (k-means, KNN, logistic regression, PyTorch loop)
3. Advanced	Statistical testing and causal inference, XGBoost internals, calibration and uncertainty, recommender and ranking systems, Transformers, MLOps, system design, implementing attention/NMS from scratch	2 (second half), 9 (second half), 12, 17, 19, 20, 21 (attention, NMS, AUC)

Tip: For each answer, practise explaining it in 30 seconds, then be ready to go one level deeper (the math, a trade-off, or a real example from your projects).

19. MLOps, Deployment & Efficiency

Q1. What is MLOps and what does an ML lifecycle look like?

Practices that combine ML, DevOps and data engineering to reliably build, deploy and maintain models. Lifecycle: data ingestion/validation → feature engineering → training & experiment tracking → evaluation & validation → model registry → deployment → monitoring → retraining (CI/CD/CT: continuous training). Tools: MLflow, Kubeflow, Azure ML, SageMaker, Vertex AI, DVC, Airflow, Feast, Weights & Biases.

Q2. Batch vs real-time (online) vs streaming vs edge inference?

Batch: scheduled predictions stored for later (nightly churn scores); simple, high throughput. **Online**: REST/gRPC endpoint with low latency per request (fraud check at payment). **Streaming**: process events continuously (Kafka + Flink). **Edge**: run on device for privacy/latency/offline (mobile, IoT) using compressed models.

Q3. What is data drift vs concept drift? How do you monitor them?

Data (covariate) drift: P(X) changes (new user demographics). **Concept drift**: P(Y|X) changes (fraud patterns evolve). **Label/prior drift**: P(Y) changes. Monitor input feature distributions (PSI, KL/JS divergence, KS test, Wasserstein), prediction distributions, data quality (nulls, schema), and actual performance when labels arrive. Respond with alerts, retraining (scheduled or triggered), and fallbacks.

Q4. What is training-serving skew?

Differences between training and production data or feature computation (different code paths, time windows, preprocessing), causing degraded performance. Prevent with shared feature pipelines, feature stores, logging production features for training, and data validation.

Q5. What is a feature store?

A centralized system to define, compute, store and serve features consistently for training (offline store, point-in-time correct joins) and inference (low-latency online store). Enables reuse and avoids training-serving skew (Feast, Tecton, Databricks, SageMaker Feature Store).

Q6. How do you ensure reproducibility in ML?

Version code (Git), data (DVC, lakeFS, snapshots), and models (registry); track experiments, hyperparameters and metrics; fix random seeds; pin dependency versions and use containers; log environment and hardware details; automate pipelines.

Q7. What deployment strategies are used for models?

Shadow deployment (new model runs on live traffic without affecting users), canary release (small % of traffic), A/B testing, blue-green deployment (switch between environments, quick rollback), multi-armed bandit routing. Always keep rollback paths.

Q8. How do you reduce model size and latency?

Quantization (INT8/INT4), pruning (unstructured weights or structured channels/heads), knowledge distillation, efficient architectures (MobileNet, DistilBERT), operator fusion and compilers (ONNX Runtime, TensorRT, OpenVINO, TorchScript/torch.compile), batching requests, caching, hardware acceleration (GPU/TPU/NPU), and early-exit models.

Q9. What is ONNX?

Open Neural Network Exchange: an open format to represent models so they can move between frameworks (PyTorch, TensorFlow, scikit-learn) and run in optimized runtimes (ONNX Runtime) across hardware.

Q10. How would you serve a model as an API?

Wrap the model in a web service (FastAPI/Flask, TorchServe, Triton, BentoML, KServe), load the model once at startup, validate inputs (schema), batch where possible, containerize with Docker, deploy on Kubernetes/managed endpoints with autoscaling, add authentication, logging, metrics (latency, throughput, errors), and health checks.

Q11. What should you monitor for a production ML model?

Operational: latency, throughput, errors, resource usage, cost. Data: schema, missing values, feature distributions, drift. Model: prediction distribution, confidence, performance metrics vs ground truth, fairness across segments. Business KPIs.

Q12. What is CI/CD/CT in machine learning? What is a model registry?

CI: on every change, run code tests plus data validation (schema, ranges), feature unit tests and quick training checks. **CD**: automatically package and deploy the model service once it passes evaluation gates (e.g., beats the current production model on a holdout set). **CT** (continuous training): retrain automatically on a schedule or when drift is detected. A **model registry** stores versioned models with metadata, metrics, lineage and stage (staging/production/archived), enabling approval workflows and rollback.

20. Scenario-Based & System Design Questions

Q13. Your model performs great offline but poorly in production. What do you check?

Data leakage in offline evaluation; training-serving skew in feature computation; data/concept drift; different preprocessing or library versions; label definition mismatch; offline metric not aligned with business metric; selection bias in training data; latency timeouts causing fallbacks; bugs in input parsing. Compare logged production features against training distributions and replay production data through the offline pipeline.

Q14. Design a fraud detection system.

Extremely imbalanced, adversarial, and real-time. Features: transaction amount, velocity (counts in last 1h/24h), device/IP, geolocation mismatch, merchant risk, user history aggregates, graph features (shared devices/cards). Model: gradient boosting baseline + rules; possibly graph/sequence models. Handle imbalance with class weights and threshold tuning; optimize precision at a fixed recall or expected cost. Time-based validation. Low-latency online feature store. Delayed labels (chargebacks) → monitor drift, retrain frequently; human review queue for medium-risk scores; feedback loop.

Q15. You have very little labeled data. What do you do?

Use transfer learning / pretrained models; data augmentation; few-shot or zero-shot LLM prompting; semi-supervised learning and pseudo-labeling; active learning to label the most valuable samples; weak supervision (labeling functions, Snorkel); synthetic data generation; simpler models with strong regularization; cross-validation for reliable estimates.

Q16. Your dataset is too big to fit in memory. How do you train?

Stream data in mini-batches (data loaders, generators), use online/incremental learners (SGD, `partial_fit`), sample intelligently, use efficient formats (Parquet, memory mapping), out-of-core tools (Dask, Spark MLlib, Vaex), distributed training, or reduce features/precision.

Q17. Training loss decreases but validation loss increases. What's happening and what do you do?

Overfitting. Use early stopping, regularization (dropout, weight decay), augmentation, more data, smaller model, check for train/validation distribution mismatch or leakage between them.

Q18. Loss becomes NaN during training. What could cause it?

Learning rate too high, exploding gradients, log(0) or division by zero in custom losses, FP16 overflow/underflow without loss scaling, bad input data (NaN/inf values), and unstable operations (use log-softmax, BCEWithLogits). Fix with gradient clipping, lower LR, numerical-stable functions, data checks.

Q19. Your model's performance has plateaued. How would you improve it?

1. **Error analysis**: inspect misclassified samples, slice performance by segment, look at the confusion matrix to find patterns.
2. **Data**: fix label noise, collect more or more representative data, augment, rebalance classes.
3. **Features**: engineer domain features, interactions, aggregations; remove noisy or leaky ones.
4. **Model**: try stronger algorithms, tune hyperparameters, use transfer learning, ensemble diverse models.
5. **Objective**: make sure the loss and metric match the business goal; tune the decision threshold.
6. Check for underfitting vs overfitting with learning curves to decide which direction to go.

Q20. How do you handle unseen categories or missing features at inference time?

Design for it during training: map rare and unseen categories to an "Other/Unknown" bucket (e.g., `OneHotEncoder(handle_unknown='ignore')`), use hashing or embeddings with an unknown token, impute missing values with the same fitted imputer and add missing-indicator flags, and validate inputs against a schema. Log and monitor how often this happens, since a rise signals data drift or upstream bugs.

Q21. How do you detect and mitigate bias (unfairness) in a model?

Sources: historical bias in data, under-representation of groups, biased labels, proxy features (zip code standing in for ethnicity), feedback loops. **Detect**: evaluate metrics sliced by group and compare fairness measures such as demographic parity (equal positive rates), equal opportunity (equal recall/TPR) and equalized odds (equal TPR and FPR). **Mitigate**: collect more representative data, reweight or resample, review proxy features, add fairness constraints during training, or adjust thresholds per group; document with model cards and monitor after deployment. Fairness definitions can conflict, so choose with stakeholders.

Q22. How would you explain a complex model to non-technical stakeholders?

Focus on business impact and decisions, not algorithms. Use simple analogies, global drivers (top features via SHAP), concrete example predictions with reasons, confidence/limitations, error trade-offs in business terms (cost of false positives vs negatives), and visualizations.

Q23. Design a search ranking / recommendation system for an e-commerce site.

Clarify objectives (CTR, conversion, revenue, diversity). Data: queries, impressions, clicks, purchases, item catalog. Architecture: candidate retrieval (BM25 + two-tower embeddings + ANN) → ranking (learning-to-rank with GBDT/LambdaMART or deep model on user, item, query and context features) → re-ranking (business rules, diversity, freshness). Handle position bias in click data, cold start, latency budgets. Offline metrics: NDCG, MRR, recall@k; online A/B test on business KPIs.

Q24. Design a customer churn prediction system.

Define churn precisely and the prediction horizon; build snapshot-based training data with features computed only from data before the snapshot (usage trends, support tickets, billing, tenure, engagement). Model with gradient boosting; evaluate with PR-AUC and lift/top-decile capture; calibrate probabilities; use SHAP for reasons to guide retention actions; consider uplift modeling to target customers who can be persuaded; deploy as batch scoring with monitoring.

Q25. How would you choose between a simple model and a complex model?

Compare on validation performance relative to business value, latency and cost constraints, interpretability/regulatory needs, data size, maintenance burden and robustness. If the complex model's gain is marginal, prefer the simpler one.

Q26. How do you handle a model whose predictions degrade over time?

Set up monitoring for drift and performance; investigate root cause (data pipeline bug vs genuine drift); retrain on recent data (sliding window or weighted), consider online learning, add features capturing the change, set retraining triggers, and keep a fallback model or rules.

Q27. Your model shows 99% accuracy. Is it a good model?

Not necessarily. Check class balance (a majority-class baseline may also score 99%), look at per-class precision/recall and the confusion matrix, check for data leakage (results that look too good), verify the split (duplicates across sets, time ordering), test on truly held-out or out-of-time data, and evaluate against the business cost of errors.

Q28. You have more features than samples (p >> n). What do you do?

The model can easily overfit and OLS has no unique solution (X^TX is singular). Use regularization (Lasso, Ridge, Elastic Net), feature selection, dimensionality reduction (PCA), simpler models, domain knowledge to drop irrelevant features, careful (nested) cross-validation, and collect more data if possible.

Q29. Training and test data come from different distributions. How do you detect and handle it?

Detect with adversarial validation: label training rows 0 and test rows 1, train a classifier; AUC near 0.5 means similar distributions, high AUC means shift, and its feature importances show which features drift. **Handle**: drop or transform shifted features, weight training samples by how test-like they are (importance weighting), build a validation set that resembles the test data, apply domain adaptation, or collect more representative data.

21. Coding Questions (Implement from Scratch)

Q30. Perform basic EDA on a dataset with pandas.

```
import pandas as pd

df = pd.read_csv("data.csv")
df.info()
df.describe(include="all")
df.isna().mean().sort_values(ascending=False)    # share of missing values per column
df.duplicated().sum()
df["target"].value_counts(normalize=True)        # class balance
df.groupby("city")["price"].agg(["mean", "median", "count"])
df.select_dtypes("number").corr()
```

Q31. Implement accuracy, precision, recall and F1 from scratch.

```
import numpy as np

def classification_metrics(y_true, y_pred):
    tp = np.sum((y_true == 1) & (y_pred == 1))
    fp = np.sum((y_true == 0) & (y_pred == 1))
    fn = np.sum((y_true == 1) & (y_pred == 0))
    tn = np.sum((y_true == 0) & (y_pred == 0))
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0.0
    accuracy = (tp + tn) / len(y_true)
    return accuracy, precision, recall, f1
```

Q32. Implement linear regression with gradient descent.

```
def linear_regression_gd(X, y, lr=0.01, epochs=1000):
    n, d = X.shape
    w, b = np.zeros(d), 0.0
    for _ in range(epochs):
        err = X @ w + b - y
        w -= lr * (2 / n) * (X.T @ err)
        b -= lr * (2 / n) * err.sum()
    return w, b
```

Scale features first; otherwise gradient descent converges slowly or diverges.

Q33. Implement logistic regression from scratch.

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def logistic_regression(X, y, lr=0.1, epochs=1000, l2=0.0):
    n, d = X.shape
    w, b = np.zeros(d), 0.0
    for _ in range(epochs):
        p = sigmoid(X @ w + b)
        w -= lr * (X.T @ (p - y) / n + l2 * w)
        b -= lr * (p - y).mean()
    return w, b

def predict(X, w, b, threshold=0.5):
    return (sigmoid(X @ w + b) >= threshold).astype(int)
```

The gradient $X^T(p - y)/n$ comes from the derivative of log loss with a sigmoid output.

Q34. Implement k-means clustering.

```
def kmeans(X, k, iters=100, seed=0):
    rng = np.random.default_rng(seed)
    centroids = X[rng.choice(len(X), k, replace=False)]
    for _ in range(iters):
        dists = np.linalg.norm(X[:, None, :] - centroids[None, :, :], axis=2)
        labels = dists.argmin(axis=1)
        new = np.array([X[labels == j].mean(axis=0) if np.any(labels == j) else centroids[j]
                        for j in range(k)])
        if np.allclose(new, centroids):
            break
        centroids = new
    return centroids, labels
```

Q35. Implement K-nearest neighbors classification.

```
from collections import Counter

def knn_predict(X_train, y_train, X_test, k=5):
    preds = []
    for x in X_test:
        dists = np.linalg.norm(X_train - x, axis=1)
        nearest = np.argsort(dists)[:k]
        preds.append(Counter(y_train[nearest]).most_common(1)[0][0])
    return np.array(preds)
```

Q36. Compute Gini impurity and find the best split for one feature.

```
def gini(y):
    _, counts = np.unique(y, return_counts=True)
    p = counts / counts.sum()
    return 1 - np.sum(p ** 2)

def best_split(x, y):
    best_t, best_score = None, np.inf
    for t in np.unique(x)[:-1]:
        left, right = y[x <= t], y[x > t]
        score = (len(left) * gini(left) + len(right) * gini(right)) / len(y)
        if score < best_score:
            best_t, best_score = t, score
    return best_t, best_score
```

Q37. Implement ROC-AUC from scratch.

```
from scipy.stats import rankdata

def roc_auc(y_true, scores):
    ranks = rankdata(scores)                      # average ranks handle ties
    n_pos = np.sum(y_true == 1)
    n_neg = len(y_true) - n_pos
    return (ranks[y_true == 1].sum() - n_pos * (n_pos + 1) / 2) / (n_pos * n_neg)
```

Uses the Mann-Whitney U statistic: AUC is the probability a random positive outranks a random negative.

Q38. Implement a numerically stable softmax and cross-entropy.

```
def softmax(z):
    z = z - z.max(axis=-1, keepdims=True)
    e = np.exp(z)
    return e / e.sum(axis=-1, keepdims=True)

def cross_entropy(logits, y):
    z = logits - logits.max(axis=1, keepdims=True)
    log_probs = z - np.log(np.exp(z).sum(axis=1, keepdims=True))
    return -log_probs[np.arange(len(y)), y].mean()
```

Subtracting the max avoids overflow in exp without changing the result (log-sum-exp trick).

Q39. Implement scaled dot-product attention.

```
def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.shape[-1]
    scores = Q @ K.swapaxes(-1, -2) / np.sqrt(d_k)
    if mask is not None:
        scores = np.where(mask, scores, -1e9)     # mask: True = may attend
    weights = softmax(scores)
    return weights @ V, weights
```

For a causal mask use `np.tril(np.ones((n, n), dtype=bool))`.

Q40. Implement IoU and Non-Max Suppression.

```
def iou(a, b):                                    # boxes as [x1, y1, x2, y2]
    x1, y1 = max(a[0], b[0]), max(a[1], b[1])
    x2, y2 = min(a[2], b[2]), min(a[3], b[3])
    inter = max(0, x2 - x1) * max(0, y2 - y1)
    area_a = (a[2] - a[0]) * (a[3] - a[1])
    area_b = (b[2] - b[0]) * (b[3] - b[1])
    return inter / (area_a + area_b - inter + 1e-9)

def nms(boxes, scores, iou_thresh=0.5):
    order = list(np.argsort(scores)[::-1])
    keep = []
    while order:
        i = order.pop(0)
        keep.append(i)
        order = [j for j in order if iou(boxes[i], boxes[j]) < iou_thresh]
    return keep
```

Q41. Build a scikit-learn pipeline with preprocessing and cross-validation.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score

preprocess = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                      ("scale", StandardScaler())]), num_cols),
    ("cat", Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                      ("onehot", OneHotEncoder(handle_unknown="ignore"))]), cat_cols),
])
clf = Pipeline([("prep", preprocess), ("model", LogisticRegression(max_iter=1000))])
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(clf, X, y, cv=cv, scoring="roc_auc")
print(scores.mean(), scores.std())
```

The pipeline refits preprocessing inside each fold, preventing leakage.

Q42. Write a PyTorch model with a training and validation loop.

```
import torch
from torch import nn

model = nn.Sequential(nn.Linear(20, 64), nn.ReLU(), nn.Dropout(0.2), nn.Linear(64, 3))
loss_fn = nn.CrossEntropyLoss()                    # expects raw logits
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-2)

for epoch in range(10):
    model.train()
    for xb, yb in train_loader:
        optimizer.zero_grad()
        loss = loss_fn(model(xb), yb)
        loss.backward()
        optimizer.step()

    model.eval()
    correct = 0
    with torch.no_grad():
        for xb, yb in val_loader:
            correct += (model(xb).argmax(dim=1) == yb).sum().item()
    print(f"epoch {epoch}: val acc = {correct / len(val_loader.dataset):.3f}")
```

— End of document. Good luck with your interviews! —